Asignatura: Computabilidad y Algoritmia
Práctica #07: Gramáticas
Nombre: Giuseppe Fuentes Moreno
Correo: alu0101644080@ull.edu.es

1. Ejercicios sobre Gramáticas independientes del contexto.

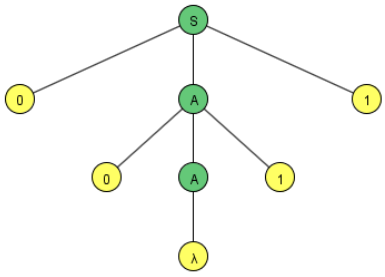
1. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{0^n 1^n \mid n \geq 1\}$.

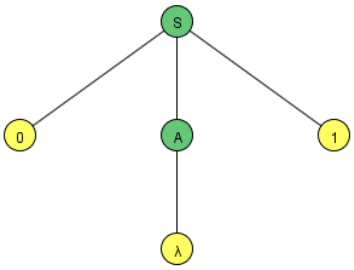
   La gramática diseñada utiliza los símbolos terminales 0 y 1, junto con las variables no terminales S (axioma inicial) y A. El criterio principal durante el diseño fue garantizar que el número de ceros y unos crezca siempre de forma simultánea y equilibrada mediante recursividad central, asegurando además que no se puedan generar cadenas vacías para cumplir estrictamente con la condición $n \geq 1$.

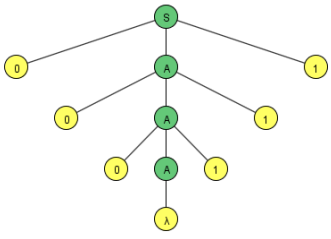
   Para lograr esto, la regla de producción de entrada $S \rightarrow 0A1$ fuerza la creación obligatoria del primer par de símbolos. A continuación, el crecimiento de la palabra se gestiona mediante la regla recursiva $A \rightarrow 0A1$, la cual permite inyectar iterativamente un "0" a la izquierda y un "1" a la derecha, manteniendo siempre la paridad exacta. El proceso finaliza gracias a la regla de parada $A \rightarrow \lambda$, que sustituye la variable restante por la cadena vacía, cerrando así la derivación una vez alcanzada la longitud deseada.

LHS		
S	⟶	0A1
A	⟶	λ
A	⟶	0A1

   Las cadenas generadas son $0011, 01, 000111$.







   Iniciamos el proceso de simplificación de la gramática con la eliminación de las producciones vacías. Como primer paso, es necesario determinar el conjunto de variables anulables $H = \{A\}$. Tras calcularlo, procedemos a ejecutar el algoritmo:
   - Eliminamos la producción original $A \rightarrow \varepsilon$
   - Analizamos la producción $S \rightarrow 0A1$. Los terminales 0 y 1 son fijos. Sin embargo, $'A'$ es anulable, por lo que debemos crear las dos combinaciones posibles: una donde $'A'$ se queda y otra donde $'A'$ desaparece.
   - Aplicamos la misma lógica en $A \rightarrow 0A1$.

   La gramática resultante es

$$S \rightarrow 0A1 \mid 01$$
$$A \rightarrow 0A1 \mid 01$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	⟶	0A1
A	⟶	0A1
S	⟶	01
A	⟶	01

Nuestra gramática no contiene producciones unitarias; por consiguiente, al intentar aplicar el algoritmo de eliminación, el conjunto de pares resultante quedaría vacío.

Tras comprobar en la primera etapa que todos los símbolos son generadores, procedemos a la segunda etapa. Dado que la única variable adicional (A) es accesible desde el axioma inicial S mediante la producción $S \rightarrow 0A1$, concluimos que la gramática carece de símbolos inútiles y se mantiene sin cambios.

La gramática final simplificada es

$$S \rightarrow 0A1 \mid 01$$
$$A \rightarrow 0A1 \mid 01$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	⟶	0A1
A	⟶	0A1
S	⟶	01
A	⟶	01

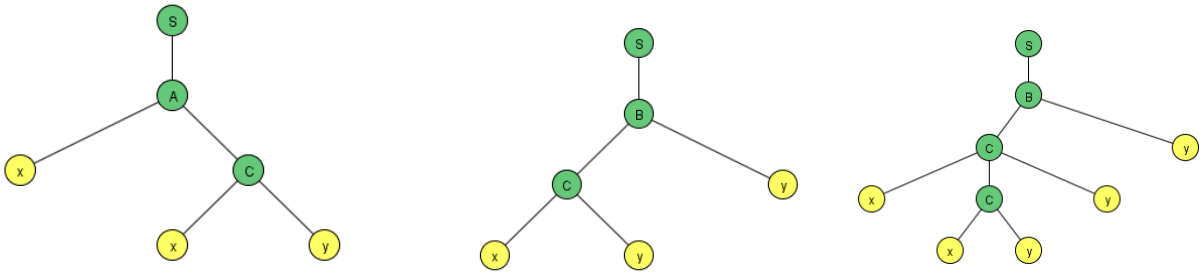
2. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{x^n y^m \mid n, m \geq 1, n \neq m\}$.

   La gramática diseñada utiliza los símbolos terminales x e y, junto con las variables no terminales S (axioma inicial), A, B y C. El criterio principal durante el diseño fue garantizar que el número de equis y griegas sea siempre distinto dividiendo la generación en dos casos lógicos ($n > m$ o $m > n$), asegurando además que haya al menos un símbolo de cada para cumplir estrictamente con la condición $n, m \geq 1$.

   Para lograr esto, la regla de producción de entrada $S \rightarrow A \mid B$ fuerza la elección obligatoria hacia el desequilibrio deseado. A continuación, el emparejamiento base de la palabra se gestiona mediante la regla recursiva $C \rightarrow xCy \mid xy$, la cual permite inyectar iterativamente una "x" a la izquierda y una "y" a la derecha, garantizando la existencia mínima de un par equilibrado. El proceso finaliza gracias a las reglas $A \rightarrow xA \mid xC$ y $B \rightarrow By \mid Cy$, que sustituyen las variables para inyectar los símbolos excedentes necesarios para romper la paridad, cerrando así la derivación con la asimetría y longitud deseadas.

LHS		
S	→	A
S	→	B
A	→	xA
A	→	xC
B	→	By
B	→	Cy
C	→	xCy
C	→	xy

Las cadenas generadas son $xxy, xyy, xxyyy$.



Iniciamos el proceso de simplificación de la gramática con la eliminación de las producciones vacías. Como carece de producciones vacías, eliminamos sus producciones unitarias. Como primer paso, es necesario determinar el conjunto de pares unitarios, obteniendo (S, A) y (S, B). Tras calcularlo, procedemos a ejecutar el algoritmo:

- Eliminamos las producciones unitarias originales $S \to A$ y $S \to B$.
- Analizamos la variable A. Añadimos a S todas las producciones no unitarias de A, es decir, xA y xC.
- Aplicamos la misma lógica en B, añadiendo a S sus producciones By y Cy.

Nuestra gramática no contiene símbolos inútiles. Tras comprobar en la primera etapa que todos los símbolos son generadores (puesto que C deriva en terminales, lo que permite que el resto también lo haga), procedemos a la segunda etapa. Dado que las variables adicionales (A, B y C) son accesibles desde el axioma inicial S mediante las producciones $S \to xA$, $S \to By$ y $S \to xC$, concluimos que la gramática se mantiene sin más cambios.

La gramática final simplificada es

$$S \to By \mid Cy \mid xA \mid xC$$
$$A \to xA \mid xC$$
$$B \to By \mid Cy$$
$$C \to xy \mid xCy$$

El resultado coincide con el obtenido por JFLAP

LHS		RHS
A	→	xA
A	→	xC
B	→	By
B	→	Cy
C	→	xCy
C	→	xy
S	→	By
S	→	Cy
S	→	xC
S	→	xA

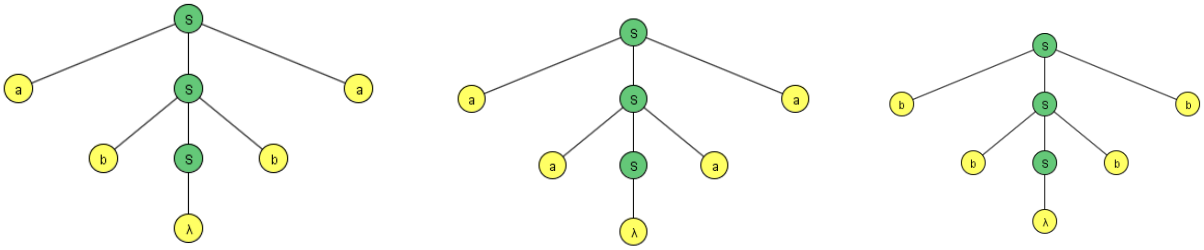
3. Diseñar una gramática independiente del contexto para el lenguaje $L = \{ww^I \mid w \in \{a, b\}^*\}$.

   La gramática utiliza los símbolos terminales a y b, junto con una única variable no terminal S, que actúa como axioma inicial. El criterio principal del diseño fue generar palabras que se lean igual de izquierda a derecha que de derecha a izquierda (palíndromos de longitud par), construyéndolas "de fuera hacia dentro" y asegurando que cada símbolo añadido a un extremo tenga siempre su réplica exacta en el extremo opuesto.

   Para lograrlo, las reglas recursivas $S \rightarrow aSa$ *y* $S \rightarrow bSb$ permiten inyectar iterativamente el mismo símbolo a la izquierda y a la derecha de la variable S, garantizando que la palabra mantenga siempre la simetría respecto a su centro. En cada paso se elige libremente entre a y b, lo que permite generar cualquier combinación de símbolos en la primera mitad de la palabra, mientras que la segunda mitad queda determinada como su reflejo. El proceso finaliza con la regla S → λ, que elimina la variable central y cierra la derivación, dando lugar a un palíndromo de longitud par (incluyendo la palabra vacía como caso base).

LHS		
S	⟶	λ
S	⟶	aSa
S	⟶	bSb

   Las cadenas generadas son $abba, aaaa, bbbb$.



   Iniciamos el proceso de simplificación de la gramática, empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de $H = \emptyset$. La producción $S \rightarrow \lambda$ es de la forma $A \rightarrow \varepsilon$, luego $H = \{S\}$. En el bucle while no hay ninguna producción $B \rightarrow A_1 \ldots A_n$ con todos los A_i en H que añada variables nuevas, por lo que $H = \{S\}$. Tras calcularlo, ejecutamos el algoritmo de eliminación:

   - Eliminamos la producción vacía $S \rightarrow \lambda$.
   - Analizamos $S \rightarrow aSa$. Como S es anulable, añadimos la versión sin S, es decir, $S \rightarrow aa$, y conservamos $S \rightarrow aSa$.
   - Aplicamos la misma lógica a $S \rightarrow bSb$, añadiendo $S \rightarrow bb$ y conservando $S \rightarrow bSb$.
   - Como $\lambda \in L(G)$, no podemos eliminar todas las producciones vacías, así que mantenemos $S \rightarrow \lambda$ como única producción vacía. La gramática resultante es $S \rightarrow \lambda \mid aSa \mid bSb \mid aa \mid bb$.

   A continuación, aplicamos la eliminación de producciones unitarias. Partimos de $H = \emptyset$. Ninguna producción tiene la forma $A \rightarrow B$ con $A, B \in V$, ya que todas tienen terminales en su parte derecha (o λ), así que no se añade ninguna pareja y $H = \emptyset$. El bucle while no se ejecuta, porque no hay parejas $(A, B)(B, C)$. No hay producciones unitarias que eliminar, y como H está vacío tampoco se añade ninguna producción nueva. La gramática se mantiene igual.

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de V′ = ∅. Las producciones $S \rightarrow \lambda, S \rightarrow aa\ y\ S \rightarrow bb$ son de la forma $A \rightarrow w$ con $w \in \Sigma^*$, luego $V' = \{S\}$. En el bucle while, las producciones $S \rightarrow aSa\ y\ S \rightarrow bSb$ tienen su único no terminal (S) en V', pero S ya está en V′, así que V′ no cambia. Como V′ = V = {S}, no se elimina ningún no terminal ni ninguna producción.
- Etapa 2: Partimos de J = {S}, V′ = {S} y Σ′ = ∅. Extraemos S de J, con lo que J = ∅.
    - S → λ no aporta terminales.
    - S → aSa aporta Σ′ = {a}, y S ya está en V′.
    - S → bSb aporta Σ′ = {a, b}, y S ya está en V′.
    - S → aa y S → bb no aportan terminales nuevos.

- Como J = ∅, el algoritmo termina con V′ = {S} y Σ′ = {a, b}, que coinciden con V y Σ. No se elimina nada.

La gramática final simplificada es

$$S \rightarrow aSa \mid bSb \mid \lambda \mid aa \mid bb$$

El resultado coincide con el obtenido por JFLAP

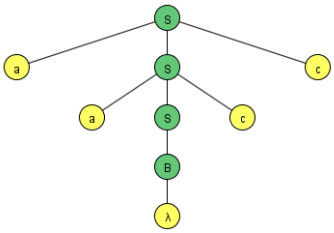
LHS		
S	→	λ
S	→	aSa
S	→	bSb
S	→	aa
S	→	bb

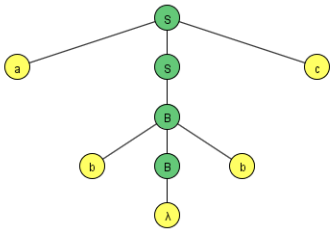
4. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m c^n \mid n \geq 0, m \text{ par}\}$.
La gramática utiliza los símbolos terminales a, b y c, junto con las variables no terminales S (axioma inicial) y B. El criterio principal del diseño fue separar la generación del lenguaje en dos partes independientes: una primera que garantiza que el número de a's y de c's sea siempre el mismo (construyéndolas "de fuera hacia dentro"), y una segunda que genera el bloque central de b's asegurando que su cantidad sea siempre par.

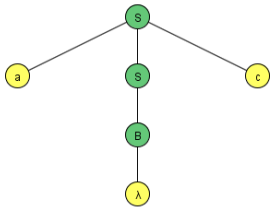
Para lograrlo, la regla recursiva S → aSc permite inyectar iterativamente una "a" a la izquierda y una "c" a la derecha de la variable S, garantizando que ambos símbolos aparezcan exactamente el mismo número de veces y que todas las a's precedan a las c's. Cuando se decide dejar de añadir pares, la regla S → B delega la generación del centro de la palabra en la variable B. Esta variable se encarga de la parte intermedia mediante la regla recursiva B → bBb, que inyecta una "b" a cada lado de B en cada paso, de modo que las b's se añaden siempre por parejas y su número resulta par. El proceso finaliza con la regla B → λ, que elimina la variable B y cierra la derivación, dando lugar a una palabra con n a's, m b's (m par) y n c's, incluyendo la palabra vacía como caso base (n = 0 y m = 0).

LHS		
S	→	aSc
S	→	B
B	→	λ
B	→	bBb

Las cadenas generadas son $ac, abbc, aacc$.







Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. La producción B → λ es de la forma $A \rightarrow \varepsilon$, luego H = {B}. En el bucle while, la producción S → B tiene todos sus símbolos (B) en H, por lo que se añade S y H = {B, S}. En la siguiente iteración, ni S → aSc ni B → bBb tienen todos sus símbolos en H, así que no se añaden variables nuevas y H = {S, B}. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- Eliminamos la producción vacía B → λ.
- Analizamos S → aSc. Como S es anulable, añadimos la versión sin S, es decir, S → ac, y conservamos S → aSc.
- Analizamos S → B. Como B es anulable, su versión sin B sería una producción vacía, que no está permitido introducir, así que solo conservamos S → B.
- Aplicamos la misma lógica a B → bBb, añadiendo B → bb y conservando B → bBb.

Como $\lambda \in L(G)$, no podemos eliminar todas las producciones vacías, así que mantenemos S → λ como única producción vacía.

La gramática resultante es

$$S \rightarrow \lambda \mid aSc \mid ac \mid B$$
$$B \rightarrow bBb \mid bb$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	λ
S	→	aSc
S	→	B
S	→	ac
B	→	bBb
B	→	bb

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de H = ∅. La producción S → B tiene la forma A → B con A, B ∈ V, luego H = {(S, B)}. El bucle while no añade nada, porque no hay parejas de la forma (A, B)(B, C): no existe ninguna pareja que empiece por B. Eliminamos la producción unitaria S → B. Para la pareja (S, B), añadimos a S todas las producciones de B, es decir, S → bBb y S → bb.

La gramática resultante es

$$S \rightarrow \lambda \mid aSc \mid ac \mid bBb \mid bb$$
$$B \rightarrow bBb \mid bb$$

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de V′ = ∅. Las producciones S → λ, S → ac, S → bb y B → bb son de la forma A → w con w ∈ Σ*, luego V′ = {S, B}. En el bucle while, las

producciones S → aSc, S → bBb y B → bBb tienen todos sus no terminales en V′, pero S y B ya están en V′, así que V′ no cambia. Como V′ = V = {S, B}, no se elimina ningún no terminal ni ninguna producción.

- Etapa 2: Partimos de J = {S}, V′ = {S} y Σ′ = ∅. Extraemos S de J, con lo que J = ∅, y recorremos sus producciones. La producción S → λ no aporta ningún terminal. La producción S → aSc aporta los terminales a y c, por lo que Σ′ = {a, c}, y como S ya está en V′ no se añade ninguna variable nueva. La producción S → ac no aporta terminales nuevos. La producción S → bBb aporta el terminal b, de modo que Σ′ = {a, b, c}, y como B no está en V′, se añade a J y a V′, quedando J = {B} y V′ = {S, B}. La producción S → bb tampoco aporta terminales nuevos.

- A continuación, extraemos B de J, con lo que J = ∅, y recorremos sus producciones: B → bBb y B → bb no aportan ni terminales ni variables nuevas, ya que B ya está en V′ y el terminal b ya está en Σ′. Como J = ∅, el algoritmo termina con V′ = {S, B} y Σ′ = {a, b, c}, que coinciden con V y Σ, por lo que no se elimina nada.

  La gramática final simplificada es

$$S \rightarrow \lambda \mid aSc \mid ac \mid bBb \mid bb$$
$$B \rightarrow bBb \mid bb$$

  El resultado coincide con el obtenido por JFLAP

LHS		
S	→	λ
S	→	aSc
S	→	ac
S	→	bBb
S	→	bb
B	→	bBb
B	→	bb

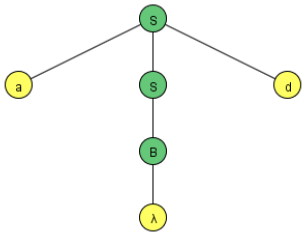
5. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m c^m d^n \mid n, m \geq 0\}$.

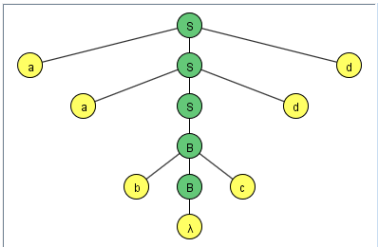
   La gramática utiliza los símbolos terminales a, b, c y d, junto con las variables no terminales S (axioma inicial) y B. El criterio principal del diseño fue separar la generación del lenguaje en dos partes dependientes de contadores distintos: un nivel exterior que garantiza que el número de a's y de d's sea siempre el mismo (construyéndolas "de fuera hacia dentro"), y un nivel interior que genera el bloque central de b's y c's asegurando que su cantidad también coincida exactamente.

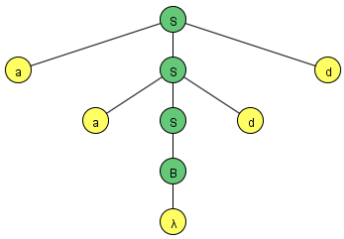
   Para lograrlo, la regla recursiva S → aSd permite inyectar iterativamente una "a" a la izquierda y una "d" a la derecha de la variable S, garantizando que ambos símbolos aparezcan exactamente el mismo número de veces en los extremos. Cuando se decide dejar de añadir pares externos, la regla S → B delega la generación del centro de la palabra en la variable B. Esta variable se encarga de la parte intermedia mediante la regla recursiva B → bBc, que inyecta una "b" a la izquierda y una "c" a la derecha de B en cada paso, de modo que las b's y c's se añaden siempre de forma simultánea y equilibrada. El proceso finaliza con la regla B → λ, que elimina la variable B y cierra la derivación, dando lugar a una palabra con n a's, m b's, m c's y n d's, incluyendo la palabra vacía como caso base (n = 0 y m = 0).

LHS		
S	⟶	B
S	⟶	aSd
B	⟶	λ
B	⟶	bBc

Las cadenas generadas son $ad, aabcdd, aadd$.







Iniciamos el proceso de simplificación de la gramática, empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. La producción B ⟶ λ es de la forma $A \rightarrow \varepsilon$, luego H = {B}. En el bucle while, la producción S ⟶ B tiene todos sus símbolos (B) en H, por lo que se añade S y H = {B, S}. En la siguiente iteración, ni S ⟶ aSd ni B ⟶ bBc tienen todos sus símbolos en H, así que no se añaden variables nuevas y H = {S, B}. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- Eliminamos la producción vacía B ⟶ λ.
- Analizamos S ⟶ B. Como B es anulable, su versión sin B sería una producción vacía, que no está permitido introducir, así que solo conservamos S ⟶ B.
- Analizamos S ⟶ aSd. Como S es anulable, añadimos la versión sin S, es decir, S ⟶ ad, y conservamos S ⟶ aSd.
- Aplicamos la misma lógica a B ⟶ bBc, añadiendo B ⟶ bc y conservando B ⟶ bBc.
- Como λ ∈ L(G), no podemos eliminar todas las producciones vacías, así que mantenemos S ⟶ λ como única producción vacía.

La gramática resultante es

$$S \rightarrow \lambda \mid B \mid aSd \mid ad$$
$$B \rightarrow bBc \mid bc$$

LHS		
S	⟶	λ
S	⟶	B
S	⟶	ad
S	⟶	aSd
B	⟶	bBc
B	⟶	bc

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de H = ∅. La producción S ⟶ B tiene la forma A ⟶ B con A, B ∈ V, luego H = {(S, B)}. El bucle while no añade nada, porque no hay parejas de la forma (A, B)(B, C): no existe ninguna pareja que empiece por B. Eliminamos la producción unitaria S ⟶ B. Para la pareja (S, B), añadimos a S todas las producciones de B, es decir, S ⟶ bBc y S ⟶ bc.

La gramática resultante es

$$S \rightarrow \lambda \mid aSd \mid ad \mid bBc \mid bc$$
$$B \rightarrow bBc \mid bc$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	⟶	λ
S	⟶	aSd
S	⟶	ad
S	⟶	bBc
S	⟶	bc
B	⟶	bc
B	⟶	bBc

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de V′ = ∅. Las producciones S → λ, S → ad, S → bc y B → bc son de la forma A → w con w ∈ Σ*, luego V′ = {S, B}. En el bucle while, las producciones S → aSd, S → bBc y B → bBc tienen todos sus no terminales en V′, pero S y B ya están en V′, así que V′ no cambia. Como V′ = V = {S, B}, no se elimina ningún no terminal ni ninguna producción.

- Etapa 2: Partimos de J = {S}, V′ = {S} y Σ′ = ∅. Extraemos S de J, con lo que J = ∅, y recorremos sus producciones. La producción S → λ no aporta ningún terminal. La producción S → aSd aporta los terminales a y d, por lo que Σ′ = {a, d}, y como S ya está en V′ no se añade ninguna variable nueva. La producción S → ad no aporta terminales nuevos. La producción S → bBc aporta los terminales b y c, de modo que Σ′ = {a, b, c, d}, y como B no está en V′, se añade a J y a V′, quedando J = {B} y V′ = {S, B}. La producción S → bc tampoco aporta terminales nuevos.

- A continuación, extraemos B de J, con lo que J = ∅, y recorremos sus producciones: B → bBc y B → bc no aportan ni terminales ni variables nuevas, ya que B ya está en V′ y los terminales b y c ya están en Σ′. Como J = ∅, el algoritmo termina con V′ = {S, B} y Σ′ = {a, b, c, d}, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática resultante es

$$S \rightarrow \lambda \mid aSd \mid ad \mid bBc \mid bc$$
$$B \rightarrow bBc \mid bc$$

El resultado coincide con el obtenido por JFLAP

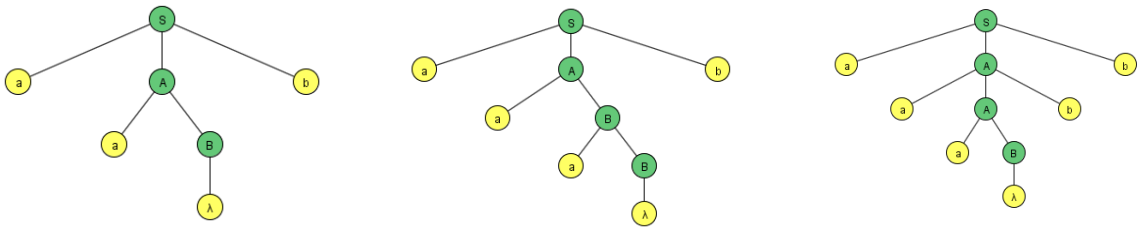
LHS		
S	⟶	λ
S	⟶	aSd
S	⟶	ad
S	⟶	bBc
S	⟶	bc
B	⟶	bc
B	⟶	bBc

6. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^n b^m \mid n > m \geq 1\}$.
La gramática utiliza los símbolos terminales a y b, junto con las variables no terminales S (axioma inicial), A y B. El criterio principal del diseño fue estructurar la generación de la palabra en tres fases secuenciales lógicas: una primera que garantiza la existencia mínima de símbolos y su emparejamiento base, una segunda que rompe el equilibrio para asegurar que haya estrictamente más a's que b's, y una tercera de crecimiento libre del excedente de a's.

Para lograrlo, la regla de producción de entrada S → aAb asegura la condición base (m ≥ 1) inyectando el primer par de símbolos obligatorios. A continuación, el crecimiento equilibrado se gestiona mediante la regla recursiva A → aAb, la cual permite inyectar iterativamente una "a" a la izquierda y una "b" a la derecha de A, manteniendo ambos contadores igualados. Cuando se decide dejar de añadir pares, se obliga a transicionar mediante la regla A → aB, que actúa como puente inyectando una "a" adicional sin su correspondiente "b", garantizando matemáticamente la asimetría requerida (n > m). Una vez en la variable B, la regla recursiva B → aB permite generar iterativamente cualquier cantidad extra de a's, ampliando libremente la diferencia entre n y m. El proceso finaliza con la regla de parada B → λ, que elimina la variable B y cierra la derivación de la palabra una vez alcanzada la longitud deseada.

LHS		
S	→	aAb
A	→	aB
A	→	aAb
B	→	aB
B	→	λ

Las cadenas generadas son $aab, aaab, aaabb$



Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. La producción B → λ es de la forma A → ε, luego H = {B}. En el bucle while no hay ninguna producción B → $A_1 \ldots A_n$ con todos los A_i en H que añada variables nuevas: S → aAb, A → aB y A → aAb contienen terminales, y B → aB también, por lo que H = {B}. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- Eliminamos la producción vacía B → λ.
- Analizamos S → aAb. Como A no es anulable, no hay ninguna versión nueva y solo conservamos S → aAb.
- Analizamos A → aB. Como B es anulable, añadimos la versión sin B, es decir, A → a, y conservamos A → aB.
- Analizamos A → aAb. Como A no es anulable, solo conservamos A → aAb.

- Analizamos B → aB. Como B es anulable, añadimos la versión sin B, es decir, B → a, y conservamos B → aB.
- Como λ ∉ L(G) (S no es anulable), se pueden eliminar todas las producciones vacías y no mantenemos ninguna.

La gramática resultante es

$$S \rightarrow aAb$$
$$A \rightarrow aB \mid a \mid aAb$$
$$B \rightarrow aB \mid a$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	⟶	aAb
A	⟶	a
A	⟶	aB
A	⟶	aAb
B	⟶	aB
B	⟶	a

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de H = ∅. Ninguna producción tiene la forma A → B con A, B ∈ V, ya que todas tienen terminales en su parte derecha, así que no se añade ninguna pareja y H = ∅. El bucle while no se ejecuta, porque no hay parejas (A, B)(B, C). No hay producciones unitarias que eliminar y, como H está vacío, tampoco se añade ninguna producción nueva. La gramática se mantiene igual.

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de V′ = ∅. Las producciones A → a y B → a son de la forma A → w con w ∈ Σ*, luego V′ = {A, B}. En el bucle while, la producción S → aAb tiene su único no terminal (A) en V′, por lo que se añade S y V′ = {A, B, S}. Las producciones A → aB, A → aAb y B → aB tienen todos sus no terminales en V′, pero A y B ya están en V′, así que V′ no cambia. Como V′ = V = {S, A, B}, no se elimina ningún no terminal ni ninguna producción.

- Etapa 2: Partimos de J = {S}, V′ = {S} y Σ′ = ∅. Extraemos S de J, con lo que J = ∅, y recorremos sus producciones. La producción S → aAb aporta los terminales a y b, por lo que Σ′ = {a, b}, y como A no está en V′, se añade a J y a V′, quedando J = {A} y V′ = {S, A}. A continuación extraemos A de J, con lo que J = ∅, y recorremos sus producciones. La producción A → aB no aporta terminales nuevos, y como B no está en V′, se añade a J y a V′, quedando J = {B} y V′ = {S, A, B}. La producción A → a no aporta nada nuevo, y la producción A → aAb tampoco, ya que A ya está en V′ y los terminales a y b ya están en Σ′. Después extraemos B de J, con lo que J = ∅, y recorremos sus producciones: B → aB y B → a no aportan ni terminales ni variables nuevas, ya que B ya está en V′ y el terminal a ya está en Σ′. Como J = ∅, el algoritmo termina con V′ = {S, A, B} y Σ′ = {a, b}, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática final simplificada es

$$S \rightarrow aAb$$
$$A \rightarrow aB \mid a \mid aAb$$
$$B \rightarrow aB \mid a$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	aAb
A	→	a
A	→	aB
A	→	aAb
B	→	aB
B	→	a

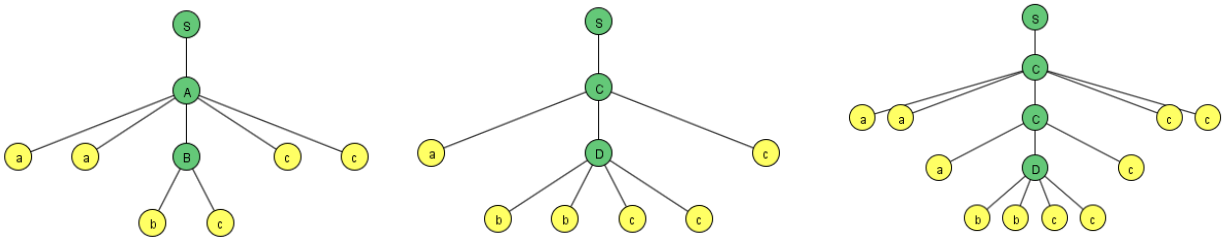
7. Diseñar una gramática independiente del contexto que genere el lenguaje $L = \{a^i b^j c^{i+j} \mid i, j \geq 1, i + j \text{ impar}\}$.
La gramática utiliza los símbolos terminales a, b y c, junto con las variables no terminales S (axioma inicial), A, B, C y D. El criterio principal del diseño fue separar la generación del lenguaje en dos caminos lógicos que garanticen la condición de que la suma total $(i + j)$ sea impar. Para que esto ocurra matemáticamente, si la cantidad de a's (i) es par, la cantidad de b's (j) debe ser impar, y viceversa. Además, se debe garantizar que por cada "a" o "b" que se añada, se inyecte exactamente una "c" al final de la palabra para cumplir con c^{i+j}, respetando el mínimo de un símbolo $(i, j \geq 1)$.

   Para lograrlo, la regla inicial S → A | C divide el proceso en estos dos casos posibles. El primer camino (variable A) asume que la cantidad de a's será par. Para ello, la regla recursiva A → aaAcc permite inyectar a's (y sus correspondientes c's) de dos en dos. Cuando se pasa a generar el bloque central, la regla de transición A → aaBcc mantiene la paridad par de a's y cede el control a la variable B. Esta variable se encarga de las b's mediante la recursividad B → bbBcc y finaliza con la regla de parada B → bc, lo que garantiza que la cantidad de b's generadas sea siempre impar.

   El segundo camino (variable C) invierte la lógica, asumiendo que la cantidad de a's será impar y la de b's será par. La variable C inyecta a's de dos en dos con C → aaCcc, pero fuerza la transición a D mediante C → aDc, garantizando así un número impar de a's (como mínimo 1). Finalmente, la variable D asume el control del centro de la palabra con la regla recursiva D → bbDcc y cierra la derivación con D → bbcc, lo que añade b's (y sus c's) estrictamente en cantidades pares, completando así la palabra con éxito.

LHS		
S	→	A
S	→	C
A	→	aaAcc
A	→	aaBcc
B	→	bc
B	→	bbBcc
C	→	aaCcc
C	→	aDc
D	→	bbDcc
D	→	bbcc

Las cadenas generadas son $aabccc, abbccc, aaabbccccc$



Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. Ninguna producción es de la forma $A \rightarrow \varepsilon$, así que H sigue siendo ∅. El bucle while no se ejecuta, porque no hay ninguna producción B → $A_1 \ldots A_n$ con todos los A_i en H. Tras calcularlo, ejecutamos el algoritmo de eliminación.

- No hay producciones vacías que eliminar.
- Como no hay variables anulables, ninguna producción genera versiones nuevas, por lo que todas se conservan tal cual.
- Como λ ∉ L(G) (S no es anulable), no es necesario mantener ninguna producción vacía.

La gramática se mantiene igual.

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de H = ∅. Las producciones S → A y S → C tienen la forma A → B con A, B ∈ V, luego H = {(S, A), (S, C)}. El bucle while no añade nada, porque no hay parejas de la forma (A, B)(B, C): no existe ninguna pareja que empiece por A ni por C. Eliminamos las producciones unitarias S → A y S → C. Para la pareja (S, A), añadimos a S todas las producciones de A, es decir, S → aaAcc y S → aaBcc. Para la pareja (S, C), añadimos a S todas las producciones de C, es decir, S → aaCcc y S → aDc.

La gramática resultante es

$$S \rightarrow aaAcc \mid aaBcc \mid aaCcc \mid aDc$$

$$A \rightarrow aaAcc \mid aaBcc$$

$$B \rightarrow bc \mid bbBcc$$

$$C \rightarrow aaCcc \mid aDc$$

$$D \rightarrow bbDcc \mid bbcc$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	aaAcc
S	→	aaBcc
S	→	aaCcc
S	→	aDc
A	→	aaAcc
A	→	aaBcc
B	→	bc
B	→	bbBcc
C	→	aaCcc
C	→	aDc
D	→	bbDcc
D	→	bbcc

Por último, aplicamos la eliminación de símbolos y producciones inútiles:

- Etapa 1: Partimos de $V' = \emptyset$. Las producciones $B \rightarrow bc$ y $D \rightarrow bbcc$ son de la forma $A \rightarrow w$ con $w \in \Sigma^*$, luego $V' = \{B, D\}$. En el bucle while, las producciones $S \rightarrow aaBcc$ y $A \rightarrow aaBcc$ tienen su único no terminal (B) en V', por lo que se añaden S y A, y la producción $C \rightarrow aDc$ tiene su único no terminal (D) en V', por lo que se añade C, quedando $V' = \{B, D, S, A, C\}$. En la siguiente iteración, el resto de producciones ($S \rightarrow aaAcc$, $S \rightarrow aaCcc$, $S \rightarrow aDc$, $A \rightarrow aaAcc$, $B \rightarrow bbBcc$, $C \rightarrow aaCcc$ y $D \rightarrow bbDcc$) tienen todos sus no terminales en V', pero las variables de su parte izquierda ya están en V', así que V' no cambia. Como $V' = V = \{S, A, B, C, D\}$, no se elimina ningún no terminal ni ninguna producción.

- Etapa 2: Partimos de $J = \{S\}$, $V' = \{S\}$ y $\Sigma' = \emptyset$. Extraemos S de J, con lo que $J = \emptyset$, y recorremos sus producciones. La producción $S \rightarrow aaAcc$ aporta los terminales a y c, por lo que $\Sigma' = \{a, c\}$, y como A no está en V', se añade a J y a V', quedando $J = \{A\}$ y $V' = \{S, A\}$. La producción $S \rightarrow aaBcc$ no aporta terminales nuevos, y como B no está en V', se añade a J y a V', quedando $J = \{A, B\}$ y $V' = \{S, A, B\}$. La producción $S \rightarrow aaCcc$ tampoco aporta terminales nuevos, y como C no está en V', se añade, quedando $J = \{A, B, C\}$ y $V' = \{S, A, B, C\}$. La producción $S \rightarrow aDc$ no aporta terminales nuevos, y como D no está en V', se añade, quedando $J = \{A, B, C, D\}$ y $V' = \{S, A, B, C, D\}$. A continuación extraemos A de J y recorremos sus producciones: $A \rightarrow aaAcc$ y $A \rightarrow aaBcc$ no aportan ni terminales ni variables nuevas. Después extraemos B de J: la producción $B \rightarrow bc$ aporta el terminal b, por lo que $\Sigma' = \{a, b, c\}$, y la producción $B \rightarrow bbBcc$ no aporta nada nuevo. Extraemos C de J: $C \rightarrow aaCcc$ y $C \rightarrow aDc$ no aportan nada nuevo. Finalmente extraemos D de J, con lo que $J = \emptyset$: $D \rightarrow bbDcc$ y $D \rightarrow bbcc$ tampoco aportan nada nuevo. Como $J = \emptyset$, el algoritmo termina con $V' = \{S, A, B, C, D\}$ y $\Sigma' = \{a, b, c\}$, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática final resultante es

$$S \rightarrow aaAcc \mid aaBcc \mid aaCcc \mid aDc$$

$$A \rightarrow aaAcc \mid aaBcc$$

$$B \rightarrow bc \mid bbBcc$$

$$C \rightarrow aaCcc \mid aDc$$

$$D \rightarrow bbDcc \mid bbcc$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	aaAcc
S	→	aaBcc
S	→	aaCcc
S	→	aDc
A	→	aaAcc
A	→	aaBcc
B	→	bc
B	→	bbBcc
C	→	aaCcc
C	→	aDc
D	→	bbDcc
D	→	bbcc

8. Diseñar una gramática independiente del contexto que genere expresiones booleanas formadas por las constantes true y false, y los operadores AND, OR y NOT. Se permitirá el uso de paréntesis para agrupar expresiones. Ejemplos: true AND false, NOT (true OR false), true AND (false OR true).

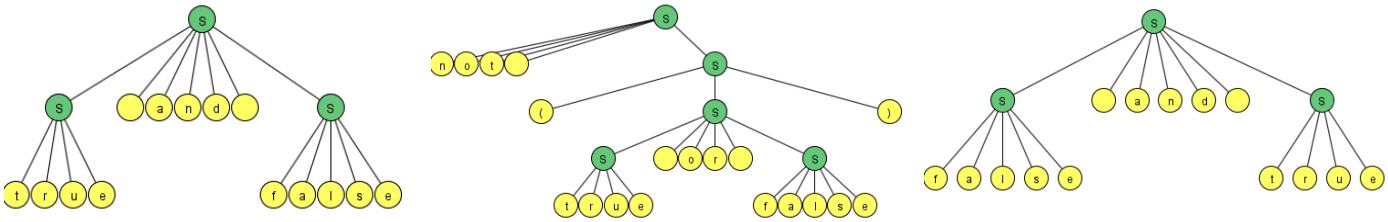
   La gramática utiliza los símbolos terminales true, false, and, or, not, (y), junto con una única variable no terminal S, que actúa como axioma inicial. El criterio principal del diseño fue agrupar todas las opciones sintácticas (operadores, agrupaciones y constantes) en una misma variable recursiva. Esto permite combinar y encadenar los elementos de forma libre, garantizando la correcta formación estructural de cualquier expresión booleana válida.

   Para lograrlo, las reglas recursivas binarias S → S and S y S → S or S permiten ramificar la expresión en dos subexpresiones evaluables, posibilitando el encadenamiento continuo de condiciones lógicas. Por su parte, la regla unaria S → not S permite negar cualquier expresión o valor directamente. Para gestionar la prioridad y agrupación estructural, se introduce la regla S → (S), que encapsula recursivamente otras subexpresiones dentro de paréntesis.

   El proceso de derivación finaliza mediante las reglas de parada S → true y S → false. Estas producciones actúan como los casos base del lenguaje, sustituyendo las variables S restantes por los valores booleanos constantes, cerrando así la derivación y dando lugar a expresiones finales evaluables como true and false o not (true or false).

LHS		
S	⟶	S and S
S	⟶	S or S
S	⟶	not S
S	⟶	(S)
S	⟶	true
S	⟶	false

Las cadenas generadas son true and false, not (true or false), false and true



Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de $H = \emptyset$. Ninguna producción es de la forma $A \to \varepsilon$, así que H sigue siendo $\emptyset$. El bucle while no se ejecuta, porque no hay ninguna producción $B \to A_1 \ldots A_n$ con todos los A_i en H. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- No hay producciones vacías que eliminar.
- Como no hay variables anulables, ninguna producción genera versiones nuevas, por lo que todas se conservan tal cual.
- Como $\lambda \notin L(G)$ (S no es anulable), no es necesario mantener ninguna producción vacía.

La gramática se mantiene igual.

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de $H = \emptyset$. Ninguna producción tiene la forma $A \to B$ con $A, B \in V$, ya que todas tienen terminales en su parte derecha (o más de un símbolo), así que no se añade ninguna pareja y $H = \emptyset$. El bucle while no se ejecuta, porque no hay parejas (A, B)(B, C). No hay producciones unitarias que eliminar y, como H está vacío, tampoco se añade ninguna producción nueva. La gramática se mantiene igual.

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de $V' = \emptyset$. Las producciones $S \to$ true y $S \to$ false son de la forma $A \to w$ con $w \in \Sigma^*$, luego $V' = \{S\}$. En el bucle while, las producciones $S \to$ S and S, $S \to$ S or S, $S \to$ not S y $S \to$ (S) tienen su único no terminal (S) en V', pero S ya está en V', así que V' no cambia. Como $V' = V = \{S\}$, no se elimina ningún no terminal ni ninguna producción.

- Etapa 2: Partimos de J = {S}, V′ = {S} y Σ′ = ∅. Extraemos S de J, con lo que J = ∅, y recorremos sus producciones. La producción S → S and S aporta los terminales a, n, d y el espacio (␣), ya que en JFLAP cada carácter es un símbolo terminal, por lo que Σ′ = {␣, a, n, d}, y como S ya está en V′ no se añade ninguna variable nueva. La producción S → S or S aporta los terminales o y r, de modo que Σ′ = {␣, a, n, d, o, r}. La producción S → not S aporta el terminal t (n y o ya estaban), con lo que Σ′ = {␣, a, n, d, o, r, t}. La producción S → (S) aporta los terminales (y), quedando Σ′ = {␣, a, n, d, o, r, t, (,)}. La producción S → true aporta los terminales u y e (t y r ya estaban), de modo que Σ′ = {␣, a, n, d, o, r, t, (,), u, e}. La producción S → false aporta los terminales f, l y s (a y e ya estaban), de forma que Σ′ = {␣, a, n, d, o, r, t, (,), u, e, f, l, s}. Como J = ∅, el algoritmo termina con V′ = {S} y Σ′ = {␣, a, n, d, o, r, t, (,), u, e, f, l, s}, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática resultante es

$$S \rightarrow true \mid false \mid S\ and\ S \mid S\ or\ S \mid not\ S \mid (S)$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	S and S
S	→	S or S
S	→	not S
S	→	(S)
S	→	true
S	→	false

9. Diseñar una gramática independiente del contexto que genere expresiones condicionales con la estructura IF condición THEN expresión ELSE expresión. La condición será una constante booleana (true o false), y las expresiones de las ramas serán números enteros no negativos. Ejemplos: IF true THEN 1 ELSE 0, IF false THEN 2 ELSE 3.

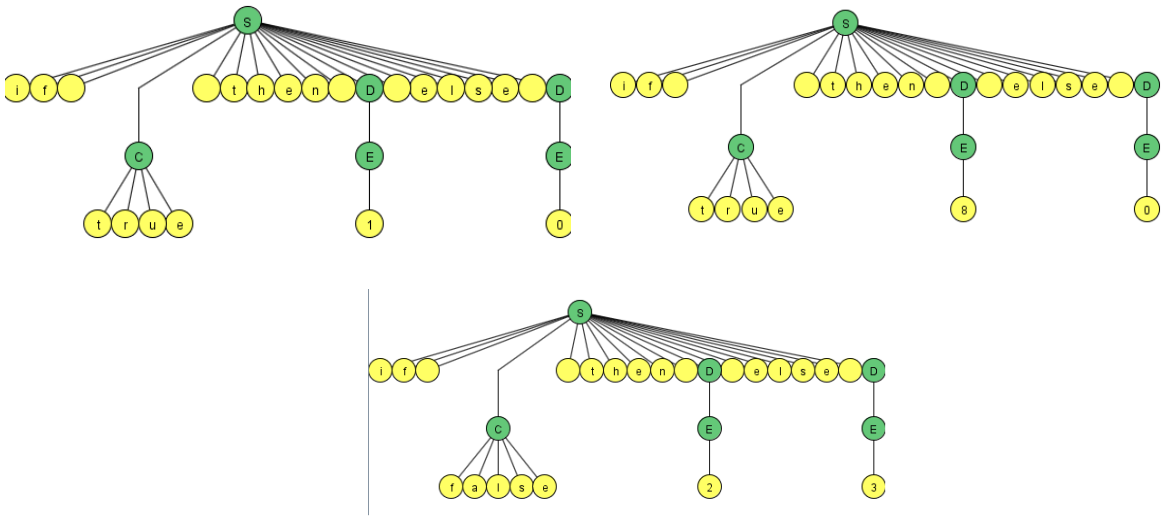
   La gramática utiliza los símbolos terminales if, then, else, true, false y los dígitos numéricos del 0 al 9, junto con las variables no terminales S (axioma inicial), C, D y E. El criterio principal del diseño fue establecer un esqueleto estructural estricto y secuencial para la sentencia condicional, delegando la generación de los elementos dinámicos (la condición y los resultados numéricos) en variables no terminales independientes para evitar que se mezclen, incorporando además la capacidad de generar números de múltiples cifras.

   Para lograrlo, la regla de producción de entrada S → if C then D else D garantiza la correcta sintaxis de la estructura, inyectando las palabras reservadas obligatorias en el orden exacto exigido por el lenguaje. Esta regla delega la evaluación lógica en la variable C, que se encarga exclusivamente de la condición booleana mediante sus reglas de derivación directa C → true y C → false. Por otro lado, los resultados de ambas ramas del condicional se gestionan ahora mediante la variable D, que permite la formación de números enteros no negativos de cualquier longitud. Mediante la regla recursiva D → DE, se pueden concatenar iterativamente tantos dígitos como se desee a la derecha, finalizando la recursión con el caso base D → E. Finalmente, la variable E asume el rol de instanciar cada uno de esos dígitos individuales, derivando directamente en cualquier

valor del 0 al 9. El proceso cierra la derivación dando lugar a sentencias condicionales válidas y perfectamente estructuradas capaces de manejar valores de múltiples cifras, como por ejemplo if true then 150 else 0.

LHS		
S	→	if C then D else D
C	→	true
C	→	false
E	→	0
E	→	1
E	→	2
E	→	3
E	→	4
E	→	5
E	→	6
E	→	7
E	→	8
E	→	9
D	→	E
D	→	DE

Las cadenas que genera son if true then 1 else 0, if false then 2 else 3 y if true then 8 else 0



Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. Ninguna producción es de la forma A → ε, así que H sigue siendo ∅. El bucle while no se ejecuta, porque no hay ninguna producción B → $A_1 \ldots A_n$ con todos los A_i en H. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- No hay producciones vacías que eliminar.

- Como no hay variables anulables, ninguna producción genera versiones nuevas, por lo que todas se conservan tal cual.
- Como $\lambda \notin L(G)$ (S no es anulable), no es necesario mantener ninguna producción vacía.

La gramática se mantiene igual.

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de $H = \emptyset$. La producción $D \rightarrow E$ tiene la forma $A \rightarrow B$ con $A, B \in V$, luego $H = \{(D, E)\}$. El bucle while no añade nada, porque no hay parejas de la forma (A, B)(B, C): no existe ninguna pareja que empiece por E. Eliminamos la producción unitaria $D \rightarrow E$. Para la pareja (D, E), añadimos a D todas las producciones de E, es decir, $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$.

La gramática resultante es

$$S \rightarrow if\ C\ then\ D\ else\ D$$

$$C \rightarrow true \mid false$$

$$E \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$D \rightarrow DE \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de $V' = \emptyset$. Las producciones $C \rightarrow$ true, $C \rightarrow$ false, $E \rightarrow 0$, …, $E \rightarrow 9$ y $D \rightarrow 0$, …, $D \rightarrow 9$ son de la forma $A \rightarrow w$ con $w \in \Sigma^*$, luego $V' = \{C, E, D\}$. En el bucle while, la producción $S \rightarrow$ if C then D else D tiene todos sus no terminales (C y D) en V', por lo que se añade S y $V' = \{C, E, D, S\}$. La producción $D \rightarrow DE$ también tiene todos sus no terminales en V', pero D ya está en V'. En la siguiente iteración no hay ninguna producción que añada variables nuevas, así que V' no cambia. Como $V' = V = \{S, C, D, E\}$, no se elimina ningún no terminal ni ninguna producción.
- Etapa 2: Partimos de $J = \{S\}$, $V' = \{S\}$ y $\Sigma' = \emptyset$. Extraemos S de J, con lo que $J = \emptyset$, y recorremos sus producciones. La producción $S \rightarrow$ if C then D else D aporta los terminales i, f, t, h, e, n, l, s y el espacio (␣), ya que en JFLAP cada carácter es un símbolo terminal, por lo que $\Sigma' = \{i, f, t, h, e, n, l, s, ␣\}$, y como C y D no están en V', se añaden a J y a V', quedando $J = \{C, D\}$ y $V' = \{S, C, D\}$. A continuación, extraemos C de J y recorremos sus producciones: $C \rightarrow$ true aporta los terminales r y u (los demás ya estaban), y $C \rightarrow$ false aporta el terminal a, de modo que $\Sigma' = \{i, f, t, h, e, n, l, s, ␣, r, u, a\}$. Después extraemos D de J y recorremos sus producciones: la producción $D \rightarrow DE$ no aporta terminales, y como E no está en V', se añade a J y a V', quedando $J = \{E\}$ y $V' = \{S, C, D, E\}$. Las producciones $D \rightarrow 0$, …, $D \rightarrow 9$ aportan los terminales del 0 al 9, quedando $\Sigma' = \{i, f, t, h, e, n, l, s, ␣, r, u, a, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Finalmente extraemos E de J, con lo que $J = \emptyset$, y recorremos sus producciones: $E \rightarrow 0$, …, $E \rightarrow 9$ no aportan nada nuevo, ya que los terminales del 0 al 9 ya están en Σ'. Como $J = \emptyset$, el algoritmo termina con $V' = \{S, C, D, E\}$ y Σ'

igual al conjunto de terminales de la gramática, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática resultante simplificada es

$$S \rightarrow if\ C\ then\ D\ else\ D$$

$$C \rightarrow true \mid false$$

$$E \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

$$D \rightarrow DE \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	→	if C then D else D
C	→	true
C	→	false
E	→	0
E	→	1
E	→	2
E	→	3
E	→	4
E	→	5
E	→	6
E	→	7
E	→	8
E	→	9
D	→	DE
D	→	4
D	→	5
D	→	6
D	→	7
D	→	0
D	→	1
D	→	2
D	→	3
D	→	8
D	→	9

10. Diseñar una gramática independiente del contexto que genere listas anidadas utilizando corchetes. Cada elemento puede ser un identificador o una lista, y los elementos se separan mediante comas. Ejemplos: [], [x], [x,y], [[x,y],[z]].

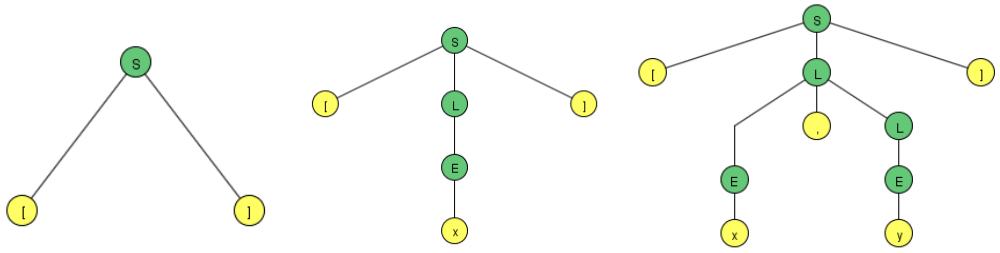
La gramática utiliza los símbolos terminales [,], ,, x, y y z, junto con las variables no terminales S (axioma inicial), L y E. El criterio principal del diseño fue estructurar la gramática separando claramente el contenedor externo (los corchetes), la secuencia de elementos (separados por comas) y el tipo de elemento individual (un identificador simple o una sublista).

Para lograrlo, las reglas iniciales S → [L] y S → [] garantizan que toda derivación comience y termine obligatoriamente con corchetes emparejados. La regla S → [] actúa como caso base directo para permitir la creación de listas vacías, mientras que S → [L] delega la generación del contenido interno a la variable L.

La variable L se encarga de gestionar la repetición y separación de los elementos mediante la recursividad L → E,L (que inyecta iterativamente un nuevo elemento seguido de una coma para continuar la secuencia) y la regla de parada L → E (que procesa el último elemento de la lista sin añadir comas al final). Finalmente, la variable E define la naturaleza de cada elemento individual: puede derivar directamente en los identificadores base mediante E → x, E → y, o E → z, o bien puede invocar de nuevo al axioma inicial mediante la regla E → S. Esta última transición es la clave del diseño, ya que permite un anidamiento recursivo perfecto y de profundidad arbitraria, logrando generar estructuras complejas como [[x,y],[z]] sin romper la sintaxis general.

LHS		
S	→	[L]
S	→	[]
L	→	E
L	→	E,L
E	→	x
E	→	y
E	→	z
E	→	S

Las cadenas que genera son [], [x] y [x,y]



Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. Ninguna producción es de la forma A → ε, así que H sigue siendo ∅. El bucle while no se ejecuta, porque no hay ninguna producción B → $A_1 \ldots A_n$ con todos los A_i en H. Tras calcularlo, ejecutamos el algoritmo de eliminación:

- No hay producciones vacías que eliminar. La producción S → [] no es vacía, ya que sus corchetes son terminales.

- Como no hay variables anulables, ninguna producción genera versiones nuevas, por lo que todas se conservan tal cual.
- Como $\lambda \notin L(G)$ (S no es anulable), no es necesario mantener ninguna producción vacía.

La gramática se mantiene igual.

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de $H = \emptyset$. Las producciones $L \rightarrow E$ y $E \rightarrow S$ tienen la forma $A \rightarrow B$ con $A, B \in V$, luego $H = \{(L, E), (E, S)\}$. En el bucle while, la pareja (L, E) junto con (E, S) forma el par (A, B)(B, C), por lo que se añade (L, S) y $H = \{(L, E), (E, S), (L, S)\}$. En la siguiente iteración no se pueden formar más parejas, así que H no cambia. Eliminamos las producciones unitarias $L \rightarrow E$ y $E \rightarrow S$. A continuación, para cada pareja añadimos las producciones no unitarias de la variable de la derecha:

- Para la pareja (L, E), añadimos a L todas las producciones de E, es decir, $L \rightarrow x \mid y \mid z$.
- Para la pareja (E, S), añadimos a E todas las producciones de S, es decir, $E \rightarrow [L] \mid []$.
- Para la pareja (L, S), añadimos a L todas las producciones de S, es decir, $L \rightarrow [L] \mid []$.

La gramática resultante es

$$S \rightarrow [L] \mid []$$

$$L \rightarrow E,L \mid x \mid y \mid z \mid [L] \mid []$$

$$E \rightarrow x \mid y \mid z \mid [L] \mid []$$

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de $V' = \emptyset$. Las producciones $S \rightarrow []$, $L \rightarrow x$, $L \rightarrow y$, $L \rightarrow z$, $L \rightarrow []$, $E \rightarrow x$, $E \rightarrow y$, $E \rightarrow z$ y $E \rightarrow []$ son de la forma $A \rightarrow w$ con $w \in \Sigma^*$, luego $V' = \{S, L, E\}$. En el bucle while, las producciones $S \rightarrow [L]$, $L \rightarrow E,L$, $L \rightarrow [L]$ y $E \rightarrow [L]$ tienen todos sus no terminales en V', pero las variables de su parte izquierda ya están en V', así que V' no cambia. Como $V' = V = \{S, L, E\}$, no se elimina ningún no terminal ni ninguna producción.
- Etapa 2: Partimos de $J = \{S\}$, $V' = \{S\}$ y $\Sigma' = \emptyset$. Extraemos S de J, con lo que $J = \emptyset$, y recorremos sus producciones. La producción $S \rightarrow [L]$ aporta los terminales [y], por lo que $\Sigma' = \{[,]\}$, y como L no está en V', se añade a J y a V', quedando $J = \{L\}$ y $V' = \{S, L\}$. La producción $S \rightarrow []$ no aporta terminales nuevos. A continuación, extraemos L de J, con lo que $J = \emptyset$, y recorremos sus producciones. La producción $L \rightarrow E,L$ aporta el terminal coma, por lo que $\Sigma' = \{[,], ,\}$, y como E no está en V', se añade a J y a V', quedando $J = \{E\}$ y $V' = \{S, L, E\}$. Las producciones $L \rightarrow x$, $L \rightarrow y$ y $L \rightarrow z$ aportan los terminales x, y y z, quedando $\Sigma' = \{[,], ,, x, y, z\}$. Las producciones $L \rightarrow [L]$ y $L \rightarrow []$ no aportan nada nuevo. Finalmente extraemos E de J, con lo que $J = \emptyset$, y recorremos sus producciones: $E \rightarrow x$, $E \rightarrow y$, $E \rightarrow z$, $E \rightarrow [L]$ y $E \rightarrow []$ no aportan ni terminales ni variables nuevas, ya que L ya está en V' y los

terminales ya están en Σ′. Como J = ∅, el algoritmo termina con V′ = {S, L, E} y Σ′ = {[,], ,, x, y, z}, que coinciden con V y Σ, por lo que no se elimina nada.

La gramática resultante simplificada es

$$S \rightarrow [L] \mid []$$

$$L \rightarrow E,L \mid x \mid y \mid z \mid [L] \mid []$$

$$E \rightarrow x \mid y \mid z \mid [L] \mid []$$

El resultado coincide con el obtenido por JFLAP

LHS		
S	⟶	[L]
S	⟶	[]
L	⟶	E,L
E	⟶	x
E	⟶	y
E	⟶	z
L	⟶	[L]
E	⟶	[]
E	⟶	[L]
L	⟶	[]
L	⟶	x
L	⟶	y
L	⟶	z

2. Ejercicios sobre Gramáticas independientes del contexto evaluación.

1. Diseña la gramática independiente del contexto que genere el siguiente lenguaje: L = {0^n 1^m | n, m ≥ 0 , n + m es impar}

La gramática independiente del contexto G esta formada por una tupla G={V,Σ,S,P}. El conjunto de símbolos terminales Σ={0,1}, no terminales V={S,A,B,C,D} y las reglas de producción siguientes:

$$S \rightarrow AB \mid CD$$
$$A \rightarrow 0 \mid 00A$$
$$B \rightarrow \varepsilon \mid 11B$$
$$C \rightarrow \varepsilon \mid 00C$$
$$D \rightarrow 1 \mid 11D$$

2.Simplifica la siguiente gramática independiente del contexto, eliminando las producciones vacías, las producciones unitarias y los símbolos inútiles. Justifica las transformaciones realizadas:

$$S \rightarrow A|B$$
$$C \rightarrow S|a|\varepsilon$$
$$B \rightarrow D|E$$
$$A \rightarrow C|D$$
$$E \rightarrow S|c|\varepsilon$$
$$D \rightarrow S|b$$

Iniciamos el proceso de simplificación de la gramática empezando por la eliminación de las producciones vacías. Como primer paso, calculamos el conjunto de variables anulables. Partimos de H = ∅. Las producciones C → ε y E → ε son de la forma A → ε, así que H pasa a ser {C, E}. El bucle while se ejecuta porque hay producciones cuyos símbolos en la parte derecha están en H: la producción A → C añade A (H = {C, E, A}); la producción B → E añade B (H = {C, E, A, B}); las producciones S → A y S → B añaden S (H = {C, E, A, B, S}); y la producción D → S añade D. Tras calcularlo, H = {C, E, A, B, S, D} (todas las variables son anulables). Ejecutamos el algoritmo de eliminación:

- Hay producciones vacías que eliminar: C → ε y E → ε.
- Como todas las variables son anulables, al intentar generar versiones nuevas de las producciones eliminando las variables anulables de su parte derecha, solo obtendríamos producciones vacías (S → ε, B → ε, etc.), las cuales no se añaden en este paso. Las producciones originales se conservan tal cual (sin las vacías).
- Como ε ∈ L(G) (S es anulable), es necesario mantener la generación de la palabra vacía añadiendo un nuevo símbolo inicial S' y las producciones S' → S | ε.

La gramática resultante es: S' → S | ε S → A | B C → S | a B → D | E A → C | D E → S | c D → S | b

A continuación, aplicamos la eliminación de producciones unitarias. Partimos de H = ∅. Las producciones S' → S, S → A, S → B, C → S, B → D, B → E, A → C, A → D, E → S y D → S tienen la forma A → B con A, B ∈ V, luego H se inicializa con todas estas parejas. En el bucle while, a través de la transitividad (pares de la forma (A, B)(B, C)), se detectan ciclos entre todas las variables originales (por ejemplo, S → A → C → S o S → B → E → S). Por lo tanto, se añaden a H todas las combinaciones posibles de parejas entre las variables {S, A, B, C, D, E}, además de todas las parejas desde S' hacia dichas variables. Eliminamos todas las producciones unitarias. A continuación, para cada pareja añadimos las producciones no unitarias de la variable de la derecha:

- Para las parejas dirigidas a C, añadimos a su variable origen todas las producciones no unitarias de C, es decir, el terminal a.
- Para las parejas dirigidas a D, añadimos a su variable origen todas las producciones no unitarias de D, es decir, el terminal b.
- Para las parejas dirigidas a E, añadimos a su variable origen todas las producciones no unitarias de E, es decir, el terminal c.
- Para el caso de S', este conserva además su producción no unitaria S' → ε.

Al estar todas las variables interconectadas por producciones unitarias, todas heredan las producciones de las demás. La gramática resultante es: S' → a | b | c | ε S → a | b | c A → a | b | c B → a | b | c C → a | b | c D → a | b | c E → a | b | c

Por último, aplicamos la eliminación de símbolos y producciones inútiles.

- Etapa 1: Partimos de $V' = \varnothing$. Las producciones de S', S, A, B, C, D y E que derivan en a, b, c o ε son de la forma A → w con $w \in \Sigma^*$, luego $V' = \{S', S, A, B, C, D, E\}$. En el bucle while no se pueden añadir más variables porque ya están todas. Como $V' = V = \{S', S, A, B, C, D, E\}$, no se elimina ningún no terminal ni ninguna producción en esta etapa.
- Etapa 2: Partimos de $J = \{S'\}$, $V' = \{S'\}$ y $\Sigma' = \varnothing$. Extraemos S' de J, con lo que $J = \varnothing$, y recorremos sus producciones. Las producciones S' → a, S' → b y S' → c aportan los terminales a, b y c, por lo que $\Sigma' = \{a, b, c\}$. La producción S' → ε no aporta terminales nuevos. Como en la parte derecha de las producciones de S' ya no aparece ninguna variable (solo terminales y ε), no se añade ningún no terminal a J ni a V′. Como $J = \varnothing$, el algoritmo termina con $V' = \{S'\}$ y $\Sigma' = \{a, b, c\}$. Dado que las variables S, A, B, C, D y E no están en V′ (son inalcanzables desde el símbolo inicial), se eliminan por completo junto con todas sus producciones.

La gramática resultante simplificada es: S' → a | b | c | ε